

ChatService.java

```
package com.harisudhan.harisudhanmart.service;

import com.harisudhan.harisudhanmart.ai.ChatProvider;
import com.harisudhan.harisudhanmart.ai.GeminiChatProvider;
import com.harisudhan.harisudhanmart.ai.MockChatProvider;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

/**
 * Section 17: selects the ChatProvider implementation via config flag (ai.chatbot.provider),
 * applies guardrails (input length cap, per-session cache), and falls back to the mock provider
 * on any failure so the widget never surfaces a raw error (Section 11, rule 3).
 */
public class ChatService {

    private static final Logger LOG = LoggerFactory.getLogger(ChatService.class);
    private static final int MAX_MESSAGE_LENGTH = 500;
    private static final String CONTEXT =
        "HarisudhanMart is a multi-seller marketplace with buyer/seller/admin roles, cart, "
        + "checkout via mock payment, order history, and reviews.";

    private final ChatProvider provider;
    private final ChatProvider fallback = new MockChatProvider();
    // Per-session response cache for repeated identical questions (Section 17, rule 4).
    private final Map<String, String> cache = new ConcurrentHashMap<>();

    public ChatService(String providerFlag, String apiKey) {
        if ("gemini".equalsIgnoreCase(providerFlag) && apiKey != null && !apiKey.isBlank()) {
            this.provider = new GeminiChatProvider(apiKey);
        } else {
            this.provider = fallback;
        }
    }

    public String reply(String sessionId, String message) {
        if (message == null || message.isBlank()) {
            return "Ask me anything about products, orders, or shipping on HarisudhanMart.";
        }
        String trimmed = message.length() > MAX_MESSAGE_LENGTH
            ? message.substring(0, MAX_MESSAGE_LENGTH) : message;

        String cacheKey = sessionId + ":@" + trimmed.toLowerCase().trim();
        String cached = cache.get(cacheKey);
        if (cached != null) {
            return cached;
        }

        String reply;
        try {
            reply = provider.getReply(trimmed, CONTEXT);
        } catch (Exception e) {
            LOG.warn("Chat provider failed, falling back to static responses", e);
            reply = fallback.getReply(trimmed, CONTEXT);
        }
        cache.put(cacheKey, reply);
        return reply;
    }
}
```